Predicting Sentiment: The Regularized Logit

MKT 566 · Fall 2026

Davide Proserpio
USC Marshall School of Business

Before We Start

Housekeeping

- **Homework 3 is due tonight, 11:59 PM**, on Brightspace
- **Homework 4 is out today** (due Nov. 19)
- Today we train the model Tuesday promised: predicting whether a review is positive or negative, **from its words alone**
- Second half of class: the **sentiment game**, in teams, laptops out

Where Tuesday left us

Step	Status
1. Label each review positive or negative	done: the data comes labeled
2. Vectorize: the TF-IDF matrix	done Tuesday
3. Split into train and test	week 9 machinery
4. **Classify: a logit on the word columns**	**today**
5. Evaluate on the test set	today, AUC and accuracy

The number to beat: Tuesday's **dictionary** (sentimentr), which needs no training data. On today's test reviews it gets **71% accuracy**. If training a model does not beat that, why bother?

Too Many Columns

The matrix is wider than it is tall

Build Tuesday's TF-IDF matrix on the training reviews and count:

- **4,000 training reviews** (80% of the 5,000; the other 1,000 are locked away as the test set)
- The training reviews contain **37,936 distinct words**. First cleanup: drop every word seen **fewer than 10 times**; you cannot learn what "ferrari" means for sentiment from 3 examples. That leaves **6,731 word columns**
- A logit estimates **one coefficient per column**: 6,731 coefficients from 4,000 examples. Still almost **two knobs per example**

> Week 5's logit was built for a handful of columns. Nothing in it says what to do when the columns outnumber the rows. Something has to give.

What actually goes wrong: memorizing

We ran that logit, with no guardrails, on the 6,731-column matrix. Here is its report card:

	Training reviews	Test reviews (never seen)
Accuracy	**100%**	83%
AUC	**1.000**	0.909

- 100% accuracy on the training set: the model found enough rare-word combinations to label every training review **perfectly**, like a student who memorizes past exams instead of learning the material
- On new reviews it drops to 83%. The gap between the two columns **is** overfitting, week 9's trap at full scale
- And it used 2,073 words to do it: far too many to read or trust

The Fix: Regularization

The idea, in plain words

Make the model **pay a price for every coefficient it uses**. Instead of minimizing only its mistakes, it now minimizes:

$$\text{Loss} \;=\; \text{how wrong the predictions are} \;+\; \lambda \times \text{how big the coefficients are}$$

- Week 5's logit minimized only the first term; the second term is **the new penalty**
- A word now earns a coefficient **only if it improves predictions enough to justify its price**. Rare coincidences ("ferrari" appeared in 3 positive reviews) are no longer worth paying for
- **λ (lambda) sets the price.** λ = 0: no penalty, the memorizing logit from the last slide. λ huge: every coefficient gets forced to zero and the model predicts the same probability for everyone
- This is called **regularization**, and the R package that does it for the logit is **glmnet**

The formula, symbol by symbol

$$\text{Loss} \;=\; \text{DataFit} \;+\; \lambda \left[\alpha \, \|\beta\|_1 \;+\; (1-\alpha) \, \|\beta\|_2^2 \right]$$

Symbol	Name	In plain words
DataFit	the logit's loss	how wrong the predictions are on the training data (what week 5's `glm` minimized)
β	the coefficients	one per word: how much that word moves the score
$\|\beta\|_1$	sum of absolute values	the **lasso** penalty: every nonzero coefficient costs
$\|\beta\|_2^2$	sum of squares	the **ridge** penalty: big coefficients cost extra
λ	penalty strength	the price dial: 0 = free, huge = everything forced to zero
α	the mix	1 = pure lasso, 0 = pure ridge, in between = elastic net

> Two dials to set, and only two: **α** (which penalty) and **λ** (how strong).

Dial 1, α: lasso or ridge?

	Lasso (α = 1)	Ridge (α = 0)	Elastic net (0 < α < 1)
What it does to coefficients	pushes many to **exactly zero**	shrinks all toward zero, none exactly	a mix of both
What you get	a **short list** of words that matter	every word keeps a small say	a compromise
Pick it when	you believe **few** features carry the signal	you believe **many** features each matter a little	you are not sure (α = 0.5 is a common default)

- Lasso's exact zeros are the headline: it does **feature selection for free**. Out of thousands of words, it hands you the few hundred worth keeping, and you can read them
- Today we use **lasso (α = 1)**: we expect a modest set of opinion words to carry most of the signal, and we want the readable shortlist

Dial 2, λ: do not guess, cross-validate

- λ = 0 memorizes; λ = huge predicts the base rate for everyone. The right value is somewhere in between, and it **differs for every dataset**, so nobody picks it by hand
- The tool you already have: **cross-validation** (week 9). Try a grid of λ values; for each one, do 5-fold CV on the **training set** and record the average AUC; pick the λ that scores best
- In R this is one line: `cv.glmnet(X, y, family = "binomial", type.measure = "auc", nfolds = 5, alpha = 1)` tries about 100 values of λ and does all the folding for you

> The test set is **not touched** during any of this. It stays in the vault until the very end, exactly like week 9.

Live: Training the Model

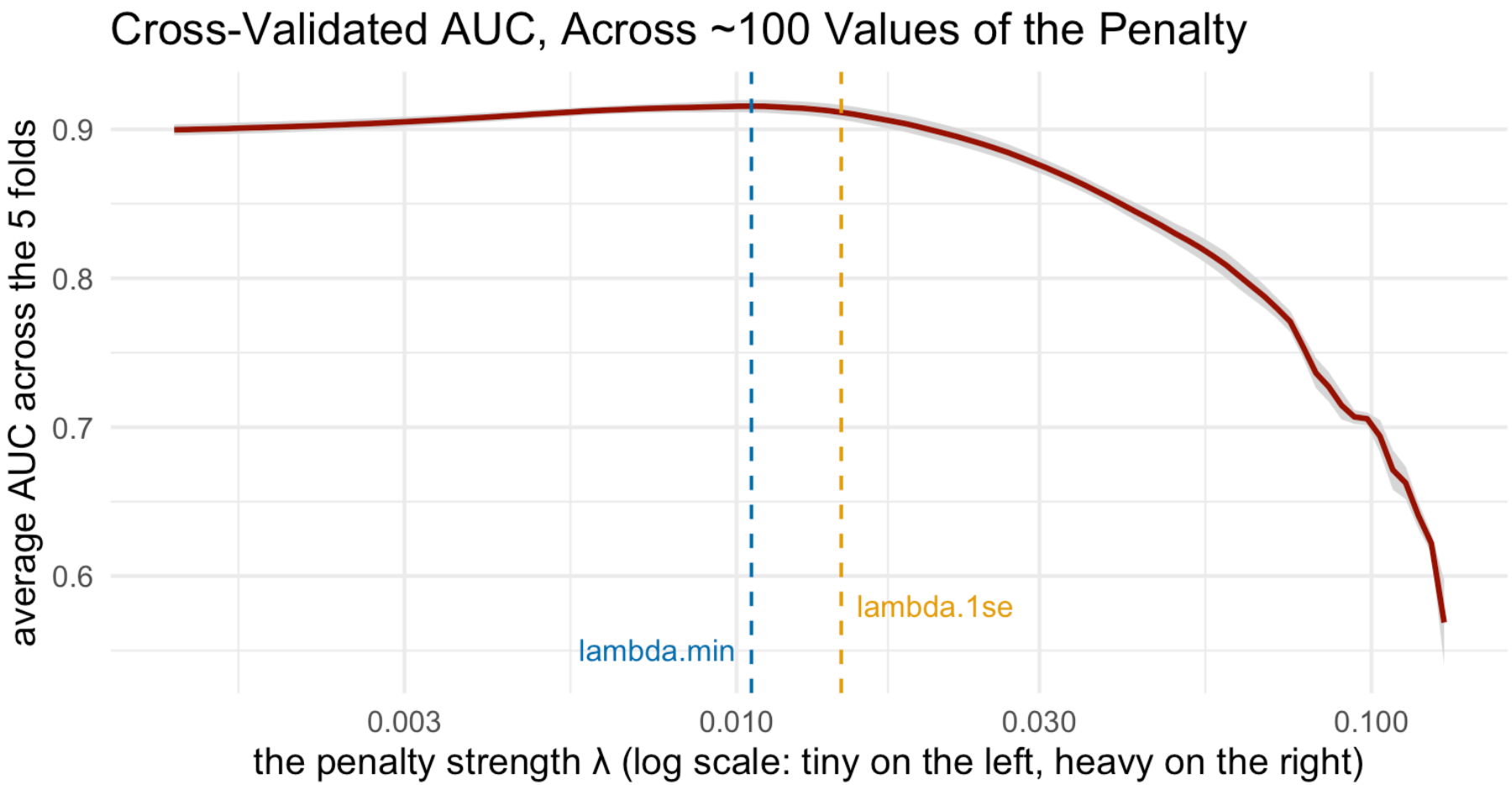
The setup, step by step

Everything below ran live when this deck was built (the class script has the code):

Step	What we did	Guardrail
1. Split	80% train (4,000), 20% test (1,000), seed 123	test set in the vault
2. Tokenize	lowercase, split into words	same as Tuesday
3. Vocabulary	all words from the **training** reviews only	the test set must not shape the features
4. Prune	drop words seen < 10 times: 37,936 → 6,731	can't learn from 3 examples
5. TF-IDF	weights **fit on train**, then applied to both	no leakage
6. Model	`cv.glmnet(...)`, lasso, 5-fold CV on AUC	λ chosen by CV, not by us

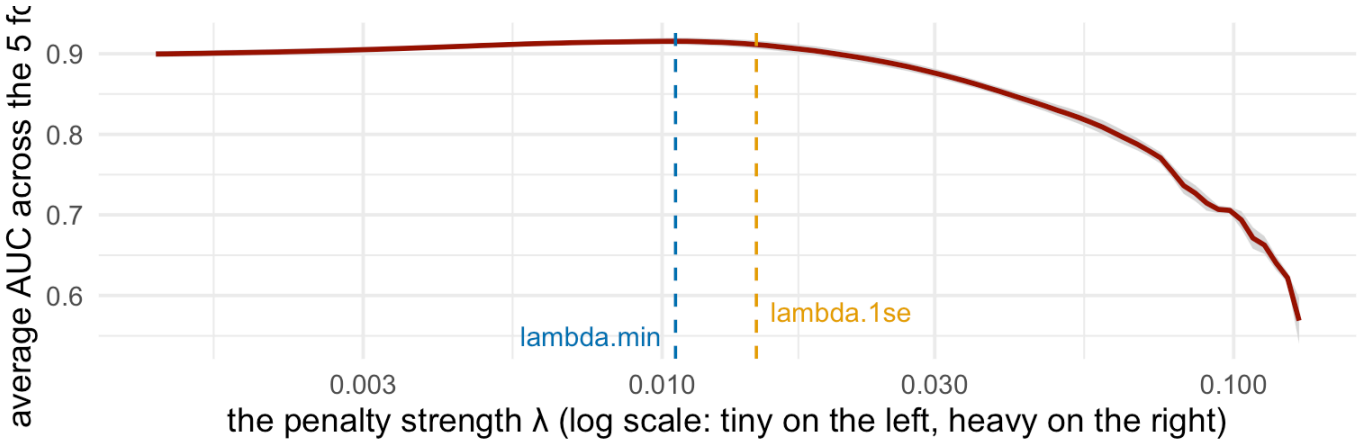
> Note how many of the guardrails are **week 9 rules**, not new text rules. Text changed the features; the discipline is unchanged.

The CV curve



One point per candidate λ: the average AUC over the 5 folds (grey band: ± one standard error). What happens at the two ends, and where would you set the dial?

Reading the CV curve



- **Left edge** (λ near zero): almost no penalty, the model memorizes, and the cross-validated AUC suffers. **Right edge** (heavy λ): the model is forced to drop everything and AUC collapses toward 0.5, a coin flip
- The top of the curve is the sweet spot. `cv.glmnet` marks two choices: **lambda.min** (blue), the λ with the best average AUC, and **lambda.1se** (gold), the heaviest penalty still within one standard error of the best: a "just as good, but simpler" option
- The sizes here: lambda.min keeps **492 words**; lambda.1se keeps only **277**. We use lambda.min today

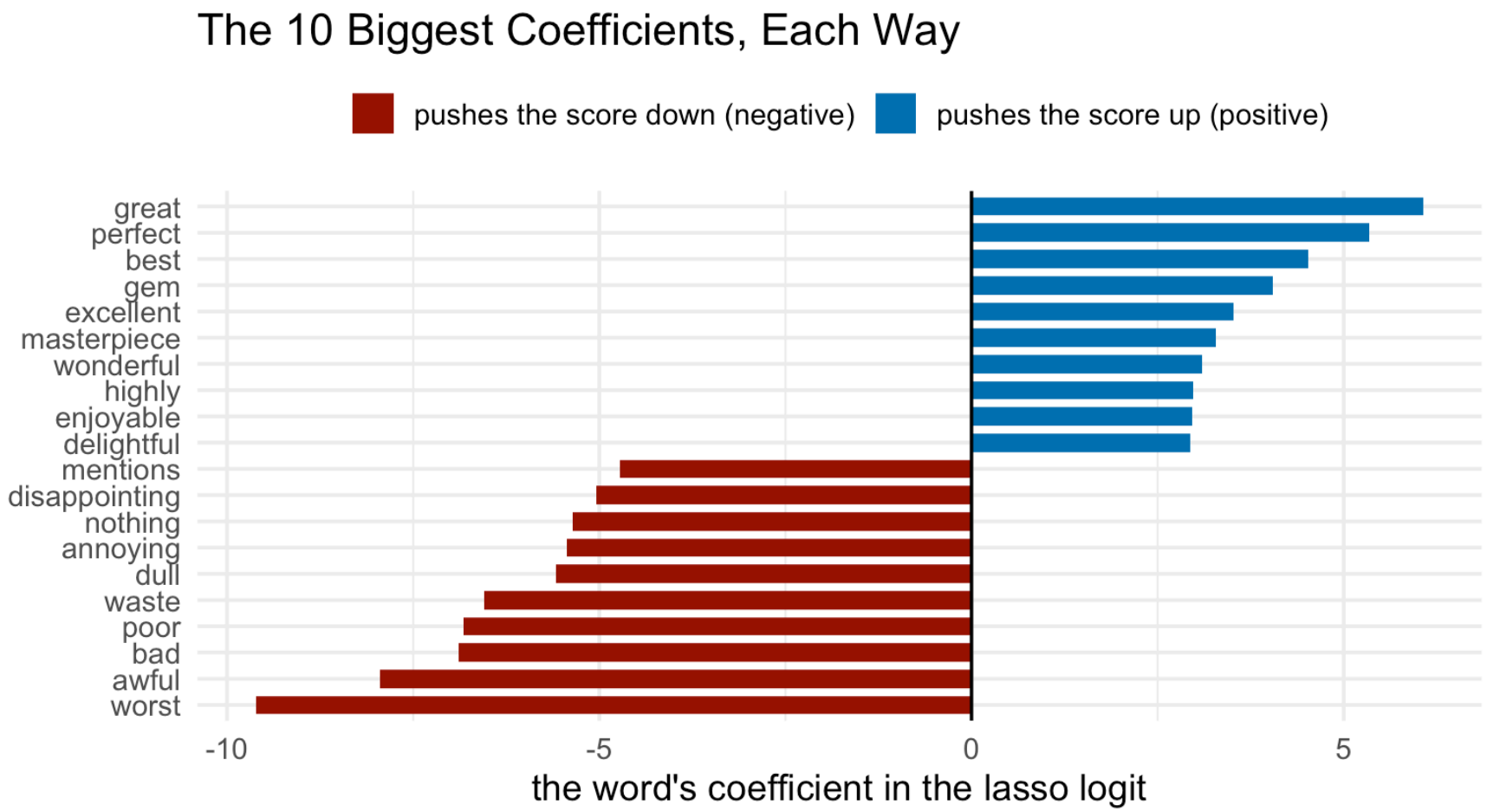
The scoreboard

All four scored on the **same 1,000 test reviews** none of them saw:

Model	Needs labeled data?	AUC	Accuracy
Coin flip	no	0.500	50%
Tuesday's dictionary (sentimentr)	no	0.824	71%
Logit, no penalty ($\lambda \approx 0$)	yes	0.909	83%
Lasso at lambda.min	yes	**0.927**	**85%**

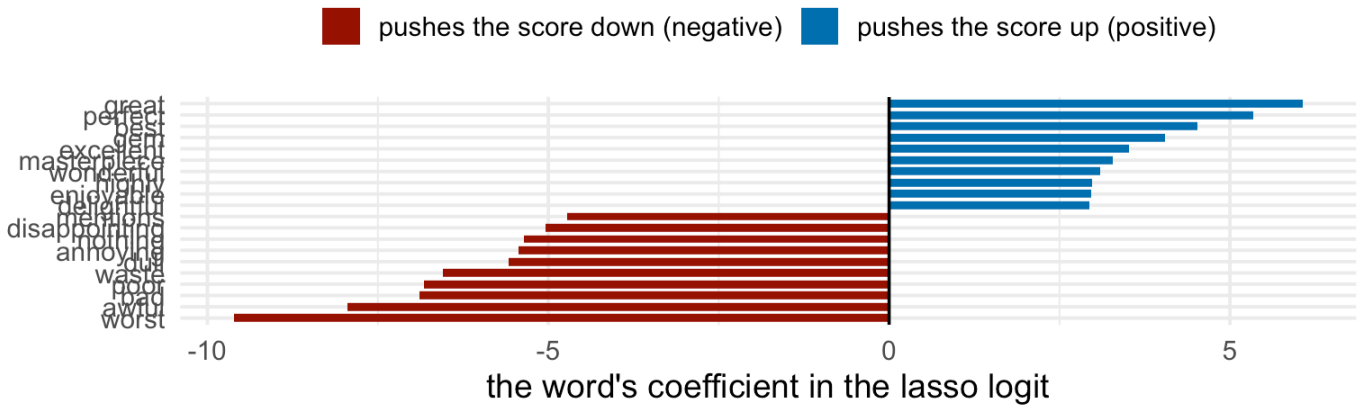
- Training on labels buys about **14 points of accuracy** over the dictionary; that is the price of the labels, paid back
- The penalty beats no penalty on every metric **and** uses 492 words instead of 2,073: better *and* smaller

The words the model trusts



The lasso's largest surviving coefficients. A review's score is, at heart, the **sum of the coefficients of the words it contains**. What do you recognize from Tuesday?

Reading the words



- These are Tuesday's counting-chart words again (**worst, awful, waste** vs. **great, perfect, excellent**), but this time **learned from the labels**, each with a weight, jointly, accounting for all other words
- Of the 6,731 words, the lasso kept 492 and set the other 6,239 to exactly zero. What survives is a shortlist a manager can read and sanity-check
- This is the quiet payoff of lasso + text: the model is **accurate and explainable at the same time**, which the random forest never was

The code: glm vs. glmnet

Week 5, the plain logit

```r
m <- glm(y ~ x1 + x2,
         data = train,
         family = "binomial")
```

- a formula and a data frame
- fine for a handful of columns
- one model back

This week, the regularized logit

```r
cv <- cv.glmnet(X_train, y_train,
                family = "binomial",
                type.measure = "auc",
                nfolds = 5, alpha = 1)
p <- predict(cv, X_test,
             s = "lambda.min",
             type = "response")
```

- a **matrix** (sparse is fine), not a formula
- built for thousands of columns
- a whole path of models, one per λ, with CV built in

The recipe, reusable

Text classification, end to end:

1. Get **labeled** examples (positive/negative, spam/not, churned/stayed)
2. Split train/test; the test set goes in the vault
3. Clean, tokenize, build the vocabulary **on the training set**, prune rare terms
4. TF-IDF, fit on train, applied to both
5. `cv.glmnet(..., family = "binomial", type.measure = "auc")`
6. Evaluate on the test set; pick the threshold the business needs (week 9)
7. **Read the surviving words**: do they make sense?

The same seven steps run spam filters, support-ticket routers, and fake-review detectors. Only step 1 changes.

The Game

The sentiment game

Teams, laptops out: **`w10-sentiment-analysis-game.R`** (also in `w10-code.zip`). Run the top of the script once, then call `run_tfidf_experiment(dt, ...)` with your own settings. One twist: in the game the labels come from **Tuesday's dictionary** (that is how teams label text when nobody has labeled it for them).

Knob	What it changes
`ngram_max = 2`	add **bigrams**: "not_good" becomes its own feature
`remove_rare`, `term_count_min`	drop rare words (we used 10 today)
`remove_popular`, `doc_proportion_max`	drop words that are in almost every review
`l2_norm`	normalize each review's TF-IDF row
`alpha`	1 = lasso, 0 = ridge, between = elastic net
`use_cv = TRUE` / `lambda_no_cv`	CV-chosen λ vs. a λ you set by hand

> **The rules:** do not touch the seed or the split. Highest **test AUC** wins; ties go to the smaller model. You have 25 minutes, then every team reports its best settings and we discuss *why* they won.

Wrap-up

The code:

- `w10-code.zip`: both class scripts + the game
- `w10-2-glmnet-class.R`: today's model, end to end, runnable line by line

The arc so far: weeks 4-5 gave you regression and the logit, week 9 taught you to **test models honestly**, and this week text became just another input. Homework 4 is out today, due Nov. 19.

Questions?